



Examen: Cotizador Birra y Bíceps con JavaScript



Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva para cotizar membresías del gimnasio y bar temático "Birra y Bíceps"**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

1. Consultar el catálogo de planes, sucursales y adicionales disponibles desde el servidor backend local.
2. Renderizar dinámicamente las opciones en los selectores y los checkboxes de servicios adicionales en el DOM.
3. Calcular en tiempo real el precio mensual y el total del contrato en base al plan elegido, recargo de sucursal, adicionales contratados y duración de la membresía con descuentos aplicados.
4. Persistir las cotizaciones contratadas en el navegador mediante `localStorage`.
5. Visualizar el historial de membresías guardadas y permitir su limpieza interactiva.



Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/servicios</code> , <code>/api/planes</code> , <code>/api/sucursales</code> , <code>/api/adicionales</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de catalogo con fetch y async await</code>
#3	Renderizar dinámicamente las opciones de planes, sucursales y los checkboxes de adicionales en el DOM.	<code>feat(js): renderizar opciones y adicionales dinamicamente en el dom</code>
#4	Implementar el cálculo reactivo del precio mensual/total y procesar la contratación con <code>submit</code> .	<code>feat(js): implementar calculo reactivo y contratacion de membresia</code>
#5	Persistir las cotizaciones en <code>localStorage</code> , mostrar el historial y permitir su limpieza con <code>#btnLimpiarHistorial</code> .	<code>feat(js): persistir y gestionar historial de cotizaciones en localStorage</code>

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/servicios` : Devuelve el objeto completo con `planes` , `sucursales` y `adicionales` .
- `GET http://localhost:3000/api/planes` : Devuelve la lista de planes.
- `GET http://localhost:3000/api/sucursales` : Devuelve la lista de sucursales con su factor de ajuste.
- `GET http://localhost:3000/api/adicionales` : Devuelve los servicios adicionales opcionales.

Para iniciar el servidor backend:

```
npm start
```

2. Reglas de Cálculo de Membresía

1. **Precio Base:** Precio del plan seleccionado $\times$ factor de la sucursal seleccionada (ej. Sede Centro: 1.10).
2. **Adicionales:** Suma directa del precio de cada adicional tildado.
3. **Precio Mensual:** `(precioPlan * factorSucursal) + sumaAdicionales` .
4. **Descuento por Duración:**
 - 1 mes: 0% descuento (`total = precioMensual * 1`).
 - 3 meses: 5% descuento (`total = (precioMensual * 3) * 0.95`).
 - 6 meses: 10% descuento (`total = (precioMensual * 6) * 0.90`).
 - 12 meses: 20% descuento (`total = (precioMensual * 12) * 0.80`).

3. Elementos Clave del DOM

- `#plan` : `<select>` para elegir el plan base.
- `#descripcionPlan` : Párrafo donde se describe el plan seleccionado.
- `#sucursal` : `<select>` para elegir la sede del gimnasio.
- `#duracion` : `<select>` con la duración en meses.
- `#adicionalesContainer` : Contenedor de checkboxes de servicios adicionales.
- `#precioMensual` y `#precioTotal` : Spans con los montos calculados.
- `#btnContratar` : Botón para procesar el submit.
- `#historialLista` : Lista `<ul>` donde se registran las cotizaciones guardadas.
- `#btnLimpiarHistorial` : Botón para vaciar el historial en `localStorage` .

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'gym_cotizaciones'`
- **Estructura:** Arreglo de objetos con `{ plan, sucursal, duracion, precioTotal, fecha }` .
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).

4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.